LESSON 11
Introduction to Next.js & CSR / SSR Concepts

WEEK 03

What is React.js?

❖ Overview

- ➢ A JavaScript library for building user interfaces, focused on components.

❖ Key Features

- ➢ Component-based, virtual DOM, declarative syntax.

❖ Use Case

- ➢ Single-page applications (SPAs).

Limitations of React.js

- **Client-Side Rendering**
  - All rendering happens in the browser, impacting initial load time.
- **SEO Challenges**
  - Search engines struggle with JavaScript-heavy apps.
- **Scalability**
  - Requires additional tools for server-side rendering.

What is Next.js?

❖ Definition

- ➢ A React framework for production-ready web applications.

❖ Key Features

- ➢ Server-side rendering, static site generation, file-based routing.

❖ Official Reference

- ➢ Next.js Homepage

Why Choose Next.js?

- **Improved Performance**
  - Optimizes rendering with SSR and SSG.
- **Better SEO**
  - Pre-rendered pages are crawlable by search engines.
- **Developer Experience**
  - Built-in routing, API routes, and TypeScript support.

Client-Side Rendering (CSR)

❖ Definition

- ➢ Rendering content in the browser using JavaScript.

❖ How it Works

- ➢ Browser downloads HTML, then JavaScript renders the UI.

Server-Side Rendering (SSR)

❖ Definition

- ➢ Rendering content on the server, sending HTML to the browser.

❖ How it Works

- ➢ Server generates HTML for each request, browser displays it.

❖ Benefits

- ➢ Faster initial load, better SEO.

Static Site Generation (SSG)

❖ Definition

- ➢ Pre-rendering pages at build time, serving static HTML.

❖ How it Works

- ➢ Pages are generated once, cached, and reused.

❖ Use Case

- ➢ Blogs, marketing pages.

Next.js Installation

- **Setup Command**
  - npx create-next-app@latest my-app
- **Project Structure**
  - Pages, public, styles, and API routes.
- **Official Guide**
  - Next.js Getting Started

File-Based Routing

❖ How it Works

- ➢ Files in pages/ become routes (e.g., pages/about.js → /about).

❖ Dynamic Routes

- ➢ [id].js for dynamic paths (e.g., /post/1).

Introduction to App Router

❖ What is App Router?

- ➢ A new routing system in Next.js 13+, replacing the Pages Router.

❖ Why Use It?

- ➢ Supports advanced features like nested layouts and server components.

❖ Official Reference

- ➢ App Router Documentation

App Router vs. Pages Router

- **Pages Router**
  - File-based routing in pages/, simpler but less flexible.
- **App Router**
  - Folder-based routing in app/, supports layouts and server components.
- **Key Difference**
  - App Router enables fine-grained control over rendering.

Setting Up App Router

❖ Project Structure

- ➢ Create an app/ folder; each subfolder represents a route.

❖ Basic File

- ➢ app/page.js defines the root route (/).

File-Based Routing in App Router

❖ How it Works

- ➢ Folders in app/ define routes; page.js in each folder is the route's content.

❖ Example

- ➢ app/about/page.js → /about.

Dynamic Routes

❖ Dynamic Segments

Use [param] folder for dynamic routes (e.g., /post/1).

```
// app/post/[id]/page.js
export default function Post({ params }) {
  return <h1>Post ID: {params.id}</h1>;
}
```

Nested Routes

- **Nested Folders**
- Subfolders create nested routes (e.g., /blog/post/1).

```
// app/blog/post/[id]/page.js
export default function BlogPost({ params }) {
  return <h1>Blog Post: {params.id}</h1>;
}
```

Layouts in App Router

- **What is a Layout?**
  - A shared UI wrapper for multiple pages.
- **File**
  - layout.js in a folder applies to all routes in that folder.

Nested Layouts

❖ How it Works

- ➢ Each folder can have its own layout.js for specific routes.

Root Layout

❖ Purpose

- ➢ Required in app/layout.js, defines the top-level HTML structure.

Server Components

❖ What are Server Components?

- ➢ Components rendered on the server by default, reducing client-side JavaScript.

❖ Benefits

- ➢ Faster initial load, better performance.

Client Components

❖ What are Client Components?

- ➢ Components requiring client-side interactivity, marked with "use client".

Combining Server and Client Components

❖ How it Works

- ➢ Server Components can render Client Components, but not vice versa.

Styling in Next.js

❖ Options

- ➢ CSS modules, Tailwind CSS, styled-components.

Image Optimization

❖ **Next.js Image Component**

➢ Automatically optimizes images for performance.

Next.js Link Component

❖ Purpose

- ➢ Client-side navigation without full page reload.

Data Fetching Strategies

- ❖ **Client-Side**
- ❖ **Server-Side**
- ❖ **Static**

Incremental Static Regeneration (ISR)

❖ What is ISR?

- ➢ Re-generate static pages in the background after a set interval.

Data Fetching in App Router

❖ Server-Side Fetching

- ➢ Fetch data directly in Server Components.

Static Site Generation (SSG)

❖ How it Works

- ➢ Pre-render pages at build time using Server Components.

Incremental Static Regeneration (ISR)

❖ Purpose

- ➢ Re-generate static pages in the background.

Loading UI

❖ **What is Loading UI?**

- ➢ Display a fallback UI while a page is loading.

❖ **File**

- ➢ loading.js in a folder.

Error Handling

❖ Error UI

- ➢ Use error.js to handle errors in a route segment.

Not Found Page

❖ Purpose

- ➢ Handle 404 errors with not-found.js.

Route Groups

❖ What are Route Groups?

- ➢ Group routes without affecting URL structure using (group) folders.

❖ Example

- ➢ app/(auth)/login/page.js → /login.

❖ Use Case

- ➢ Separate auth routes from public routes.

Metadata in App Router

❖ Purpose

- ➢ Define SEO metadata for pages.

Middleware in App Router

❖ Purpose

- ➢ Run code before rendering routes (e.g., authentication).

API Routes in App Router

❖ How it Works

- ➢ Use **route.js** in **app/api/** for serverless APIs.

TypeScript with App Router

❖ Setup

- ➢ App Router works seamlessly with TypeScript.

Performance Optimization

❖ Techniques

- ➢ Use Server Components to reduce client-side JavaScript.

Deployment with Vercel

- **Why Vercel?**
  - Built by Next.js creators, seamless integration.
- **Steps**
  - Push to GitHub, connect to Vercel, deploy.
- **Reference**
  - Vercel Deployment

Common Pitfalls

- **Overusing Client Components**
  - Increases client-side JavaScript, slowing performance.
- **Ignoring Metadata**
  - Impacts SEO if not configured.
- **Misusing Layouts**
  - Over-nesting layouts can complicate rendering.